Scope of Improvement

1. Structured Intent

Current Situation
Your Acceptance Contract is written in plain English, for example:

Produce email performance result for fiscal week 3 FY2026.

This is easy for a human reviewer to understand, but difficult for code to evaluate automatically.

Recommendation

Add a small structured intent block for important test cases.

Example:
```
expected_intent:
  topic: email_performance
  timeframe: FW3 FY2026
  metric_scope: email
```

The runner can compare the expected intent with the actual intent extracted from the generated SQL or graph state.

Why is this useful?

Instead of only checking whether the system produced an answer, you can verify whether it understood the user's business intent correctly. This makes your evaluation objective and easier to automate.

2. Automatic Semantic Scoring

What does it mean?

Automatic semantic scoring means comparing the expected meaning of the question with what the backend actually produced. It does **not** compare exact SQL—it compares the intent.

Example

Question:
How is email performing FW3 FY2026?

Expected Intent:
```
topic: email_performance
timeframe: FW3 FY2026
```

Actual Intent (from SQL/graph):
```
topic: email_performance
timeframe: FW3 FY2026
```

How scoring works

Assign points for the important parts of the intent.

• Topic correct = 30 points
• Timeframe correct = 30 points

• Metric correct = 20 points
• Entity correct = 10 points
• Safe behavior = 10 points

Total = 100 points.

Where to add it in your framework

Current flow:
Run Test → Capture Final Graph State → Write cases.jsonl

Improved flow:
Run Test → Capture Final Graph State → **Evaluate Semantic Score** → Write cases.jsonl

What to store in cases.jsonl

Store:
• Expected Intent
• Actual Intent
• Semantic Score
• Status (PASS / NEEDS REVIEW / FAIL)

Why is this important?

Today, a test may simply report **PASS**. With semantic scoring, you can explain *why* it passed or failed.

Example:
• TF01 = 100 (Correct timeframe)
• CC01 = 80 (Comparison correct, timeframe incorrect)
• OOS02 = 0 (System silently ignored an unsupported dimension)

This gives measurable NL2SQL quality instead of only human interpretation.